



Report: About applying the 10 principles of clean code to a registration program

Prepared by:

- Youssef Ashraf
- Eslam Ahmed
- Ahmed Abdelraheem
- Ahmed Mohamed Ali Galal
- Mousa Abdelbaset Mousa

Delivered to :

- ENG: Ghada Moustafa Mohamed Kadous

User Registration System (C++ Console App)

A robust, secure, and clean-code oriented console application for user registration. This project demonstrates **Separation of Concerns**, **Input Validation**, and **Basic Security Practices** using C++.

Features

- **Clean Architecture:** Code is separated into Logic, Data, and Presentation layers.
- **Robust Validation:**
 - **Name:** Accepts letters and spaces (for full names).
 - **Email:** Validates format (must contain @ and .).
 - **Phone:** Checks for digits and country code format.
 - **Password:** Enforces complexity (Letters + Digits + Special Characters).
 - **Date of Birth:** Validates logic (e.g., day 1-31, month 1-12).
- **Security:**
 - **Password Hashing:** Passwords are hashed before storage (never stored as plain text).
- **Data Persistence:** Users are saved to a central database file (users_db.txt).
- **User Experience:** Retry mechanism for invalid inputs (doesn't crash or exit unexpectedly).

Project Structure

The project follows a modular structure to ensure maintainability:

```
Registration_Form/
├── main.cpp          # Entry point (Presentation Layer) - Handles UI &
flow.
├── User.h            # Model (Entity) - Represents the User data
structure.
├── Validator.h       # Logic Layer - Contains all static validation
rules.
├── file.h            # Infrastructure Layer - Handles file I/O and
hashing.
└── users_db.txt      # Database - Stores registered users
(generated automatically).
```

Technical Details

1. The User Class (User.h)

A pure data container (POCO) that holds user attributes. It does not contain any logic, strictly adhering to the **Single Responsibility Principle (SRP)**.

2. The Validator Class (Validator.h)

A utility class containing static methods. It validates user input before object creation to ensure data integrity.

- **Why Static?** Because validation rules do not depend on a specific instance of a class.

3. The FileManager Class (file.h)

Handles the persistence layer.

- **Appending Mode:** Opens the file in `std::ios::app` to add new users without overwriting existing ones.
- **Hashing:** Implements a basic hashing mechanism using `std::hash` to secure passwords.

(Technical Report)

: User Registration System - Architecture & Implementation Report

1. Executive Summary

The objective of this project was to develop a C++ console application for user registration that adheres to **Clean Code** principles. The system focuses on data integrity through rigorous input validation and ensures basic security by hashing sensitive data. The architecture was designed to be modular, separating the user interface from business logic and data storage.

2. System Architecture

The project utilizes a **Layered Architecture** approach to ensure Separation of Concerns:

- **Presentation Layer (main.cpp):**
 - **Responsibility:** Interacts with the user via the console.
 - **Function:** It captures inputs using `std::getline` to handle spaces correctly and calls the `Validator` class. It manages the application flow (loops for retrying inputs).
- **Domain/Entity Layer (User.h):**

- **Responsibility:** Defines the core data structure.
- **Function:** Holds state (Name, Email, Phone, etc.) without behavior, ensuring a clean data model.
- **Business Logic Layer (Validator.h):**
 - **Responsibility:** Enforces business rules.
 - **Function:** Validates formats (Email syntax, Phone digits, Password complexity) independent of the UI or Storage.
- **Data Access Layer (file.h):**
 - **Responsibility:** Persists data to the file system.
 - **Function:** Handles file opening, writing, and closing. It also acts as a security gate by hashing passwords before writing them to the disk.

3. Key Technical Decisions

- **Why Separation of Concerns?**
 - By separating Validator and FileManager, the code becomes easier to test and maintain. If the validation logic needs to change (e.g., allowing longer passwords), we only modify Validator.h without touching the main file.
- **Security Implementation (Hashing):**
 - Storing passwords in plain text is a critical security vulnerability.
 - **Solution:** We implemented std::hash in the FileManager. While this is a basic hashing mechanism, it demonstrates the concept of not storing raw credentials, satisfying the security requirement for this level of the project.
- **Input Handling:**
 - A common issue in C++ is reading strings with spaces.
 - **Solution:** Replaced cin >> with std::getline(). A helper function getValidInput() was created in main to encapsulate the “prompt -> read -> validate -> retry” loop, significantly improving the User Experience (UX).

4. Validation Logic Breakdown

Field	Rule Enforced
Name	1-50 chars, starts with a letter, allows spaces (no numbers/symbols).
Email	Must contain @ and ., ensures logical position of characters.
Phone	Must be all digits. Country code must start with +.
Date	Format DD-MM-YYYY, logical ranges (Day: 1-31, Month: 1-12).
Password	8-20 chars, must include: Upper/Lower case, Digit, Special char.

5. Conclusion

The “User Registration System” successfully meets the requirements of a clean, modular, and secure C++ application. The code is reusable, readable, and scalable. Future iterations could involve adding a database engine (SQLite) and a graphical user interface (GUI).